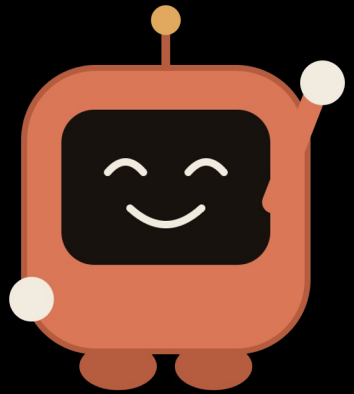
A Tester's Guide to Claude Code

From Setup to Scripting



Adonis Celestine Senior Director, Global Automation Practice Lead, Applause

Taras Lytvyn AI-first Test Automation Architect, Lead SDET, Applause



Adonis Celestine

Author, International Speaker
Global Head of Automation



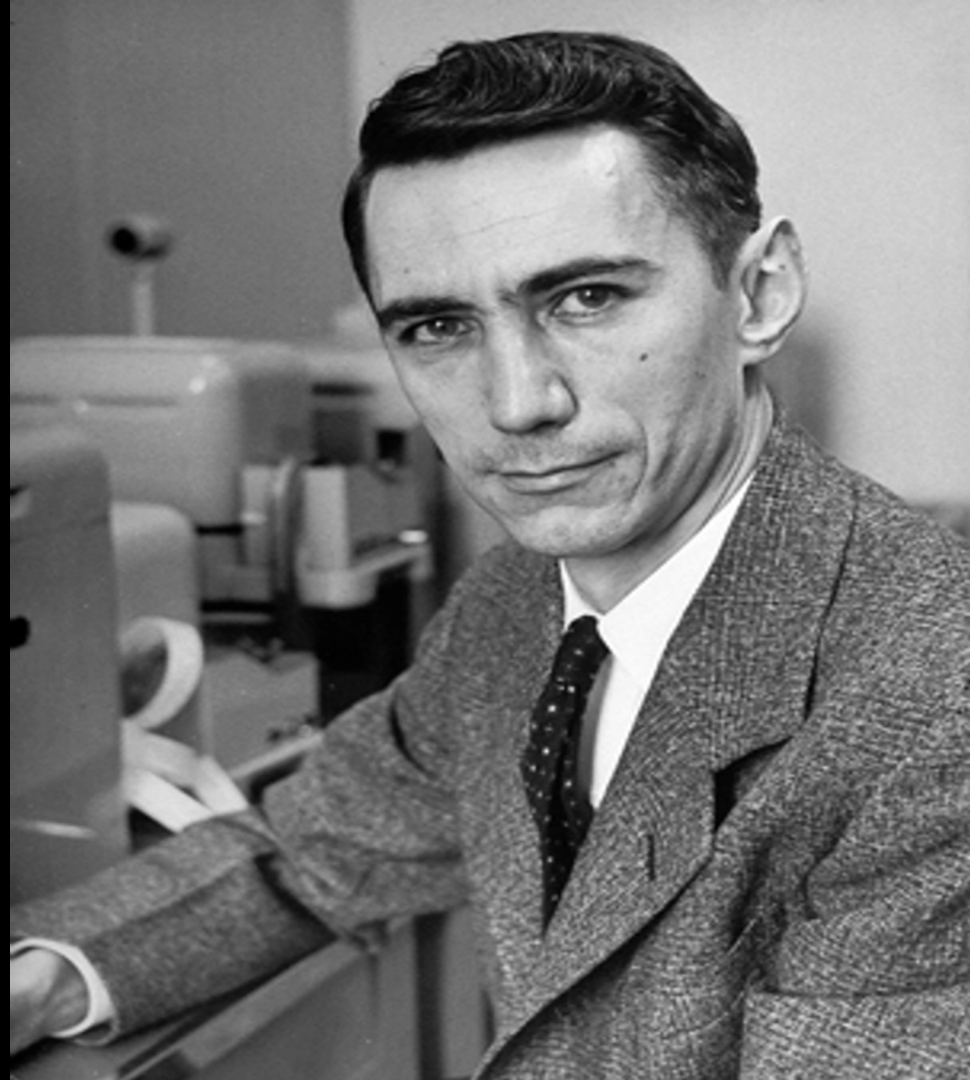
Taras Lytvyn

AI-first Test Automation Architect
Lead SDET

APPLAUSE

1.5M+	QA community
200+	Countries & territories
150+	Languages
10M+	Devices worldwide
200K+	Bugs found a year

Who is Claude named after?

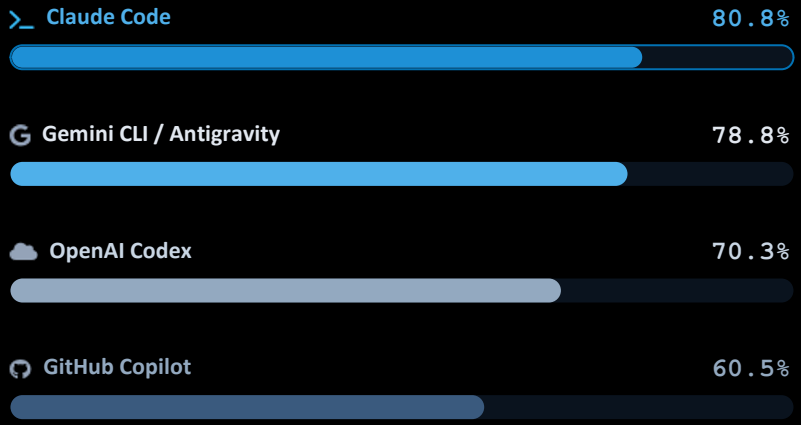


Why we chose Claude Code for test automation

SWE-bench

ISSUE SOLVE RATE

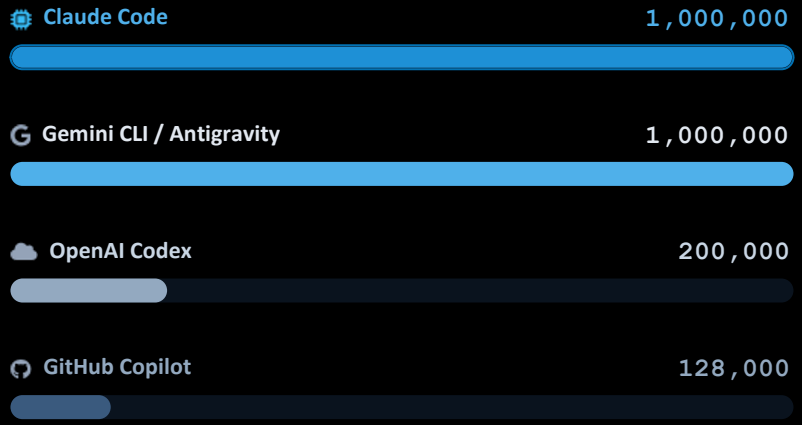
Real-world GitHub issue resolution without regression



Context Horizon

TOKEN CAPACITY

Total repository & test log working memory per session



Installation & Setup

1 INSTALL, IN YOUR TEST REPO

your-test-repo

```
# install
$ curl -fsSL https://claude.ai/install.sh | bash

# start a session
$ claude

# writes a first CLAUDE.md
> /init
```

2 WHY COMPANIES RUN IT ON THEIR OWN CLOUD

Company login

Cloud IAM or Entra ID. No personal API keys.

Your data

On Bedrock only AWS sees it. Not Anthropic.

One bill

On your cloud invoice, split by team.

Your region

You choose it, and every call is logged.

The Foundation & Building blocks

BUILDING BLOCKS



Skill

PLAYBOOK



Subagent

WORKER



Hook

GUARDRAIL



MCP Server

TOOL PLUG



Plugin

BUNDLE

RESTS UPON FOUNDATION

THE FOUNDATION

Context

WORKING MEMORY

Memory

PERSISTENT STATE



Context

Working memory: everything Claude can see right now.

KEEP IT CLEAN

`/context`	see what is filling it
`/compact [focus]`	summarise, keep what matters
`auto-compact`	runs on its own when nearly full
`/clear`	start fresh for a new task
`subagents`	read big things, return a summary

TOOLS THAT SAVE TOKENS

caveman	Terse replies, fewer tokens. Also shrinks CLAUDE.md.
Context7	Current Playwright/Appium docs for your version.
Serena	Finds code by symbol, not by reading whole files.
Repomix	Packs a repo into one AI-friendly file.



Memory

Stays on your machine. To share it, commit it: git is the sync.

WHERE IT LIVES

Organisation	`managed CLAUDE.md (by IT)`
You, all projects	`~/.claude/CLAUDE.md`
Team, one repo	`./CLAUDE.md + .claude/rules/`
Several repos	`@~/team-standards/qa.md`
Past sessions	`claude --continue / --resume`

TOOLS WE USE

claude-mem	Searchable history of past sessions. We use it daily.
mem0	Long-term memory for your own QA agents.
basic-memory	Plain Markdown notes, versioned next to the repo.

Building Blocks in Action: A QA Example

1	**MCP Server** ADDS A TOOL	**Plugs Claude into outside tools & live runtimes** Called whenever Claude needs to inspect an external app	**Playwright / Appium** Inspect real browser DOM & mobile UI
2	**Skill** SAYS HOW	**Markdown playbooks (SKILL.md) for repeatable tasks** Loaded on demand or invoked directly via /command	**/api-tests** Generates tests using team conventions
3	**Hook** ALWAYS RUNS	**Deterministic scripts triggered automatically by events** Enforces hard rules on tool calls, edits, and commands	**block rm -rf** Blocks destructive file or DB operations
4	**Subagent** WORKS ON SIDE	**Helper agent with isolated memory and tailored tools** Processes large diffs and logs without filling your context	**test-reviewer** Catches flaky sleeps & weak assertions
5	**Plugin** BUNDLES ALL	**Distributable package bundling skills, agents, hooks & MCP** Installed once with /plugin to share across team repos	**qa-toolkit** One-command team distribution

Example building block: MCP

MCP (Model Context Protocol) is a standard plug: install a server, and Claude gets new tools.

Playwright MCP
Web: a real browser, clicks, network.

Appium MCP
Android & iOS, emulators or real devices.

Mobile-mcp
Open-source iOS & Android automation.

GitHub
Issues and PRs from failed runs.

Jira & Confluence
Atlassian MCP: stories, bugs, test plans.

Clappy MCP (Applause)
Recorded flow → Playwright framework.

Use a CLI when…

- **Repeatable jobs:** tests, lint, CI.
- **Claude Code drives a browser:** playwright-cli.
- **Huge output:** filter the logs first.

Use MCP when…

- **No shell:** chat apps, IDE panels.
- **Hosted tools with login:** Jira, Confluence.
- **Live exploration:** one step at a time.

SAME BROWSER TASK, TOKENS



≈4×
fewer with the CLI

Example building block: Skills and Sub Agents

.claude/skills/api-tests/SKILL.md

```
---
name: api-tests
description: Write Playwright API tests
  (no browser) based on the team docs.
  Use when asked to write API tests.
---
1. Search the team docs first
2. Cover every error code; keep checking
   async results until they settle
3. Run them, report which doc each
   is based on
```

.claude/agents/test-reviewer.md

```
---
name: test-reviewer
description: Reviews new tests for
  hard waits, weak asserts, flakiness.
tools: Read, Grep
---
```

Works in its own working memory and reports back a short verdict.

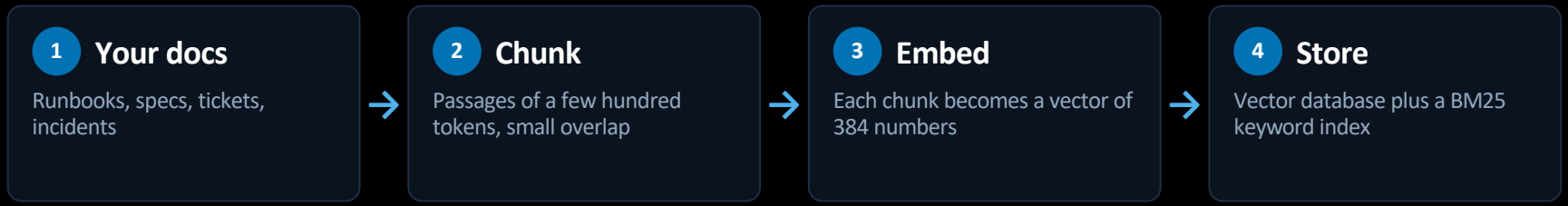
Where to get skills

1. anthropics/skills: webapp-testing (Playwright)
2. qaskills.sh: hundreds of QA skills, npx qaskills add …
3. Plugin marketplaces: /plugin

Best practices

Ground it in your docs · Give it rules

Inside RAG: how it finds the right page



BM25 KEYWORDS

Ranks chunks by matching words. Rare words count more.

```
search_docs("QA-1432 payment-status")
```

✓ **Finds** the ticket and the test with that exact ID

Best at: **exact IDs, error codes, locators**

Embeddings MEANING

Finds the chunk vectors nearest to the question.

```
search_docs("why is checkout slow?")
```

✓ **Finds** “up to 8 s at peak”, with no shared words

Best at: **synonyms, plain-English questions**

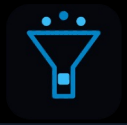
The 4 Evolution Stages of RAG



Naive RAG

Single-pass lookup. Best for querying a QA wiki or fetching a single known Jira ticket.

Fastest • Fragile with noise



Advanced RAG

Reranks CI logs + Confluence to find the exact root cause of a flaky test failure.

High Precision • Production default



GraphRAG

Maps knowledge graphs to link requirements, test coverage, and code defects.

Deep Reasoning • Gap Analysis



Agentic RAG

Autonomous testing loops: writes test, runs it, reads logs, fixes code, repeats until passing.

Iterative • Self-Healing

Start simple: no index
The Atlassian MCP server searches Confluence and Jira live.

Our demo setup
Chroma plus a small search_docs MCP server. Local and free.

Managed
Bedrock Knowledge Bases or Vertex AI RAG Engine do the indexing.

/karpathy-guidelines: one file, four rules

GET IT: TWO COMMANDS

```
> /plugin marketplace add multica-ai/andrej-karpathy-skills
> /plugin install andrej-karpathy-skills@karpathy-skills
```

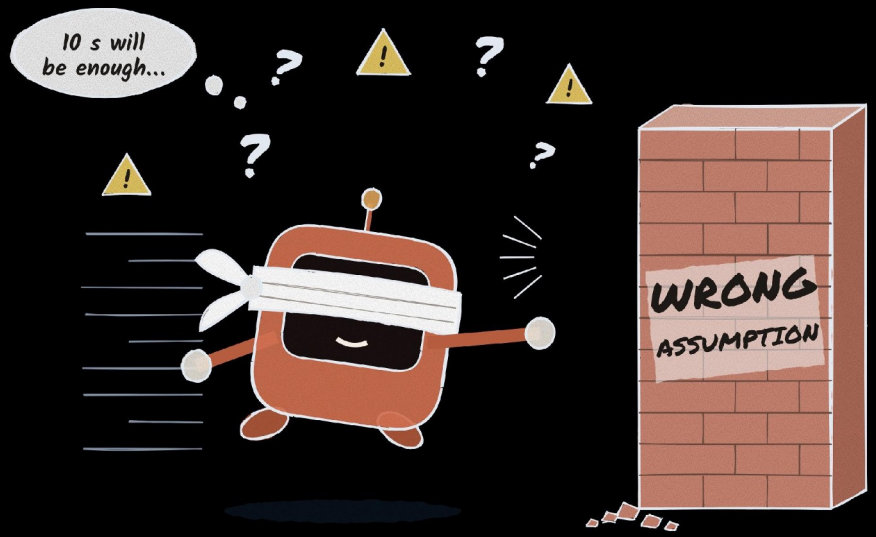
SCAN TO GET IT

github.com/multica-ai/
andrej-karpathy-skills

THE FOUR RULES, ONE PER SLIDE NEXT

1. Think first
2. Keep it simple
3. Touch only the task
4. Prove it

Problem 1 — Wrong Assumptions



Think first

State your assumptions. If something is unclear, ask before writing any code.

TESTING EXAMPLE

Task **Fix the flaky checkout test.**

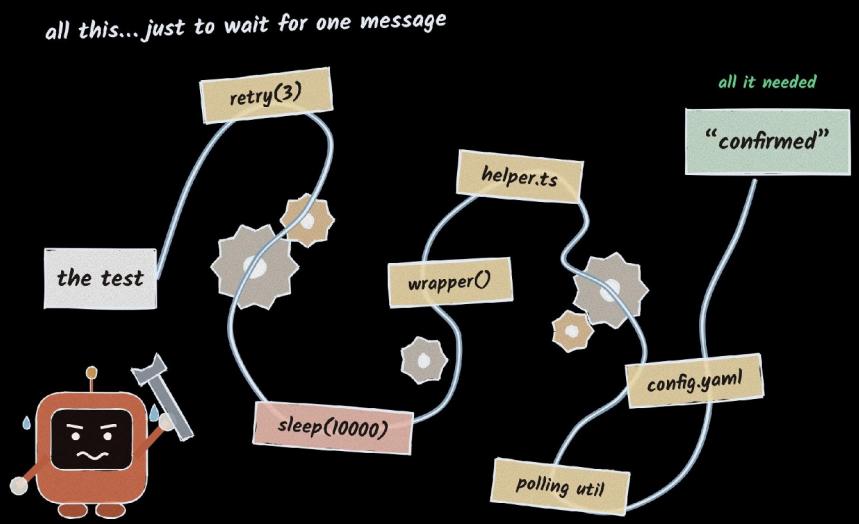
✗ **Without** Silently sets a 10 s timeout, picked from what it saw on this laptop.

✓ **With the rule** Asks first: “I don't know how fast your CI is.”

IMPACT ON YOUR TEST PROJECT

Guesses surface as questions your docs or team can answer, not as failures in CI.

Problem 2 — Over Complication



Keep it simple

Write the minimum code that solves the task. No speculative helpers, retries or options.

TESTING EXAMPLE

Task **Make the test wait for the payment.**

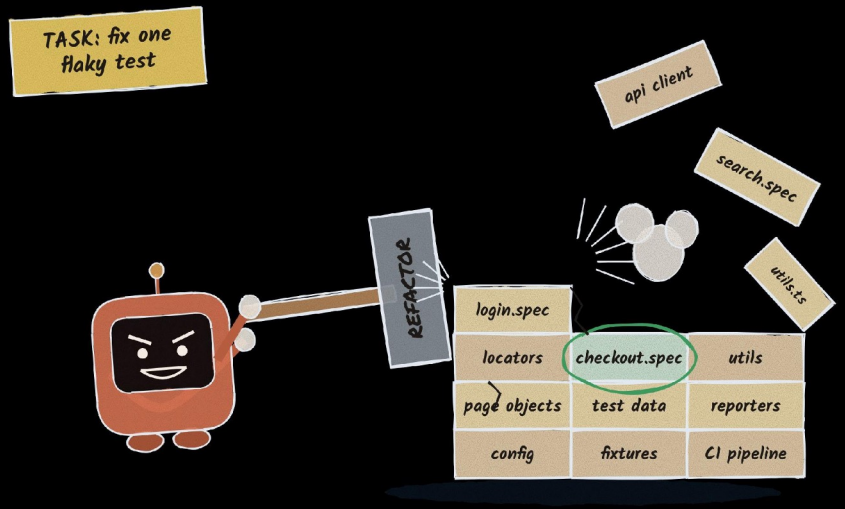
✗ **Without** A retry wrapper, sleep(10000) and a new polling util.

✓ **With the rule** Waits for the real "confirmed" response, then one retrying assertion.

IMPACT ON YOUR TEST PROJECT

Less test code to maintain, and the real cause stays visible instead of buried under retries.

Problem 3 — Collateral Damage



Touch only the task

Change only what the task needs. Match the existing style. Mention other issues, don't fix them.

TESTING EXAMPLE

Task **Fix checkout.spec.**

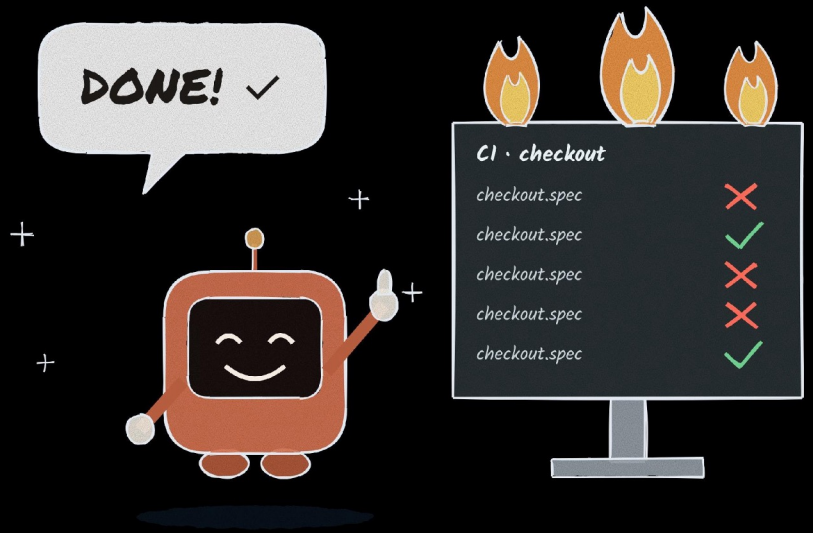
✗ **Without** Renames locators, adds a utils file, edits login.spec too.

✓ **With the rule** One test changed. Same getByRole / getByTestId locators.

IMPACT ON YOUR TEST PROJECT

Small diffs reviewers can trust, and no surprise failures in tests nobody asked to change.

Problem 4 — “Done” without proof



Prove it

Define success before you start, then show the evidence, not just “done”.

TESTING EXAMPLE

Task **Make the checkout test stable.**

✗ **Without** Says “Done!” after a single green run.

✓ **With the rule** Runs it 30 times with --repeat-each and reports “passed 30/30”.

IMPACT ON YOUR TEST PROJECT

A green build means something again: you review evidence, not claims.

PART 2

Real Problems - Real Solutions

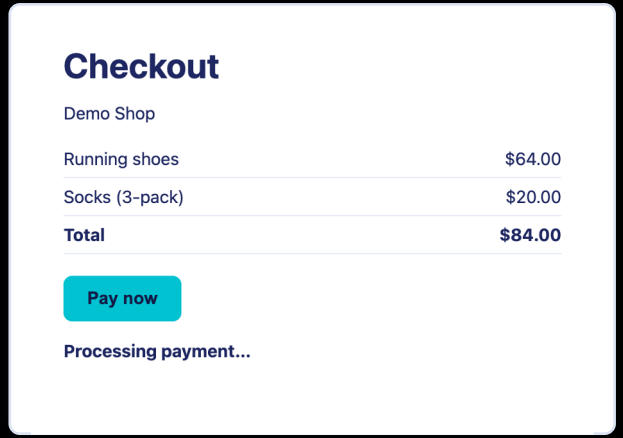
A typical QA problem: the flaky checkout test

What you see: the checkout test fails about 4 runs in 10 (Received: “Processing payment…”), then passes on a re-run

What teams usually do: add CI retries, or raise the fixed wait in the test (ticket QA-1432)

What it costs: slower releases, and nobody trusts a red build any more

Where it happens: a web shop whose payment confirms asynchronously (1–3 s, up to 8 s at peak). The app works; the test checks too early.



Fixing it with the four /karpathy-guidelines rules

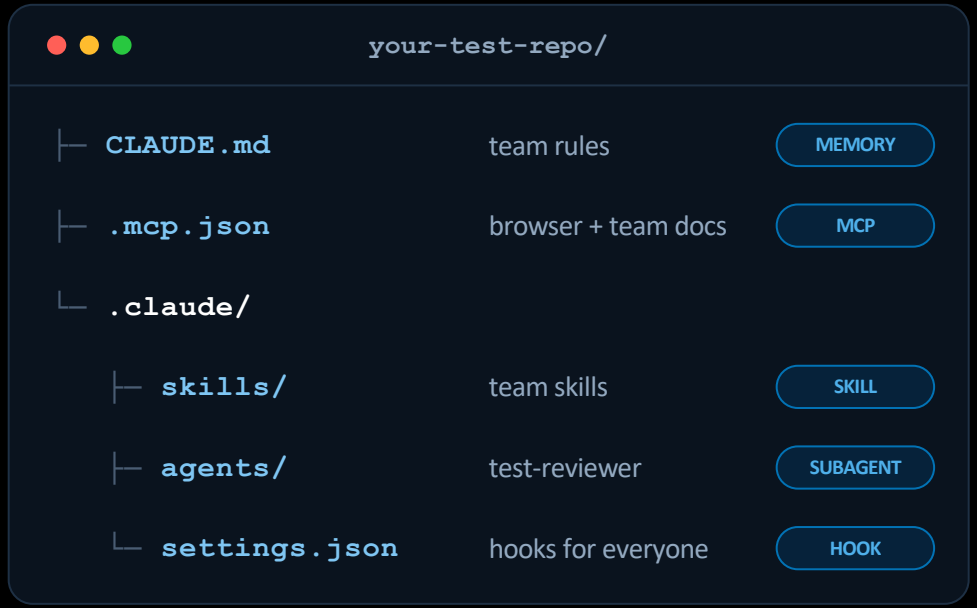
1 Think first
reproduce it, watch it, read the docs

2 Keep it simple
wait for the real "confirmed" event

3 Touch only the task
one test changed, same locators

4 Prove it
several runs in a row, also on a slow day

Step 1: Start with a good project structure



1 **Start with**

One person runs /init once in the test repo. It writes a first CLAUDE.md.

2 **Commit it**

The whole team shares the same rules, and changes them by pull request, like code.

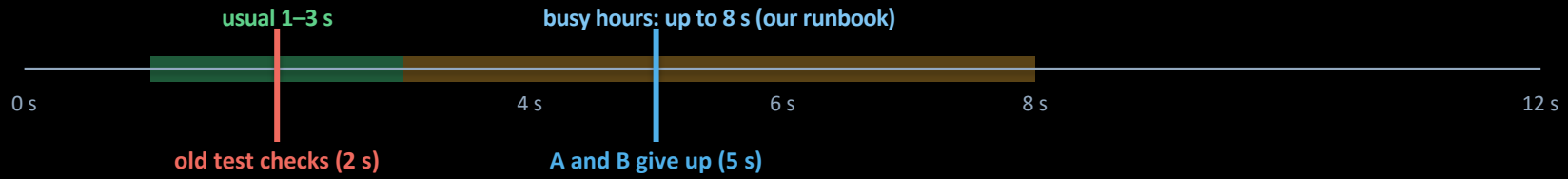
3 **Keep personal preferences out**

Each tester's own ~/.claude/CLAUDE.md, not in the repo.

Why first: every tester's Claude now starts with the same rules, skills and tools.

Step 2: without your docs, Claude can only guess

The bug: payment takes 1–8 s, but the test checks once after 2 s.
We asked Claude to fix it with no team docs, twice.



A · Plain prompt

`toHaveText(checkoutTexts.paymentDone)`

Removed the sleep (Playwright waits up to 5 s). Ran it on this laptop and said "Fixed."

✗ **Fails when payment takes 8 s.**

B · + /karpathy-guidelines

`the same change`

…and it warned us: "only 1.5 s of slack… I haven't seen CI timings. Say if you want it."

Honest, but still fails at 8 s.

Same code, different answers. Only your docs know the real answer: wait for "confirmed", not a time.

One more layer: the karpathy rules + your team's knowledge

Karpathy's four rules HOW TO WORK
think first · keep it simple · touch only the task · prove it

+

Your team's docs (RAG) WHAT IS TRUE
search_docs via MCP finds the runbook: wait for “confirmed”, up to 8 s

+

A real browser WHAT REALLY HAPPENS
Playwright MCP watches the page: Pay → 202 → pending → confirmed

=

`/fix-flaky` one command, every tester

Rules alone made Claude honest. Rules + docs made it right.

READ MORE

1. Karpathy's notes
x.com/karpathy

2. The skill file
github.com/multica-ai/...

3. Why it took off
Medium

4. Most popular AI coding skills
dev.to/aws

Step 3: write the four rules down once, as a team skill

.claude/skills/fix-flaky/SKILL.md

1. **Think first**
   Reproduce it, watch it in the browser, search the docs.
2. **Keep it simple**
   Wait for the real event, never a timeout.
3. **Touch only the task**
   Change only the test, in our existing style.
4. **Prove it**
   Run several times, name the docs you used.

NOW EVERY TESTER TYPES

```
> /fix-flaky
  the checkout test
```

Same command, same quality, for everyone on the team.

The four /karpathy-guidelines rules, written once as steps your whole team runs.

Step 6: several testers, several Claudes, one repo

All testers now use these skills at the same time. How they avoid breaking each other's work:



1

Rules via PR

Edits to CLAUDE.md, skills and hooks get reviewed like code.



2

One task, one branch

Two agents at once? claude --worktree gives each its own repo copy.



3

Same guardrails for all

Hooks (automatic checks) live in the shared settings and run for everyone.



4

Every PR is reviewed

Claude in your CI (GitHub Action) plus a human reviewer.



5

Learnings go back

A new team decision becomes a line in CLAUDE.md.

Git keeps collisions small and visible: separate branches or worktrees, small tasks. Two people editing CLAUDE.md? A normal merge conflict, solved in review.

The result: the fix waits for the real event, not a number

Before

the original test, 10 runs



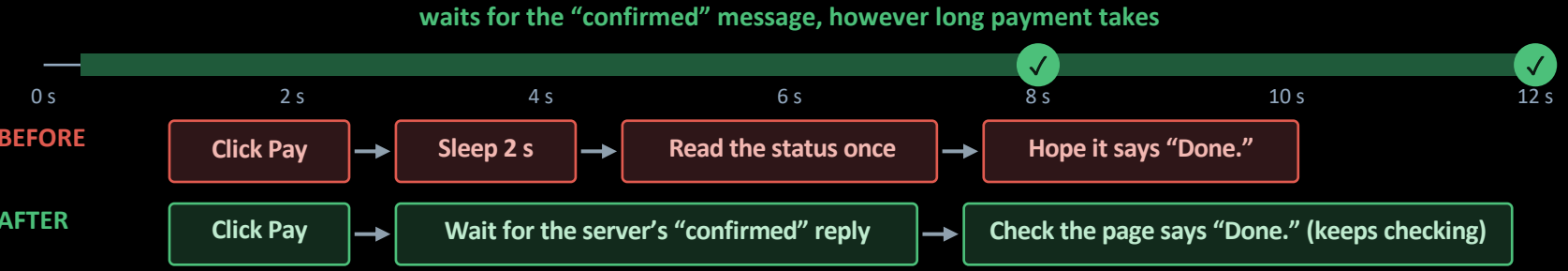
fails about 4 in every 10

After /fix-flaky

re-run by us, 10 runs



10/10 · also passes when payment takes 8 s and 12 s



Where the fix lives: the page object **pages/checkout.page.ts**. Every test calls its **pay()** method, so this one change fixes them all.

BEFORE	`await this.page.waitForTimeout(2000);`	✗ sleeps 2 s, then hopes payment is done
AFTER	`await this.page.waitForResponse(… "confirmed");`	✓ waits for the server’s real “confirmed” reply

Your repo, same result: rules in CLAUDE.md + your docs connected + a /fix-flaky skill. Every tester gets this fix.

What made the difference

WHAT CLAUDE HAD	WHAT IT DID	RESULT
Just a prompt	Removed the sleep and said “Fixed.”	✗ Fails on a busy day (8 s)
Prompt + karpathy rules	The same fix, but warned us it didn’t know the CI timings.	✗ Still fails at 8 s, but honest
karpathy rules + our docs (RAG) + browser	/fix-flaky waited for the real “confirmed” reply, in our page object, in our style.	✓ 10/10, passes at 8 s and 12 s

Rules make Claude honest. Your docs make it right.

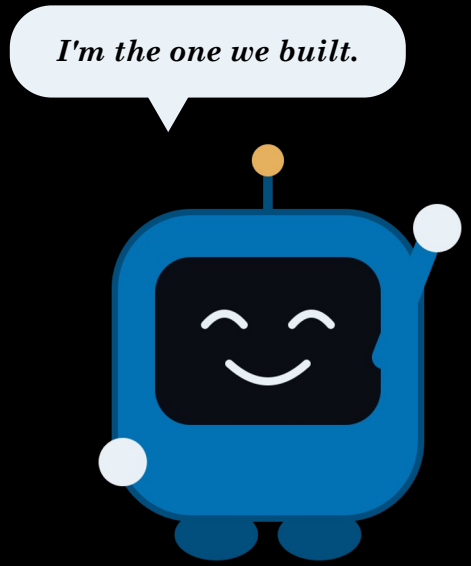
A team skill makes it the same for everyone.

PART 3

One test fixed. Now the whole team.

Clappy SDK

Meet Clappy

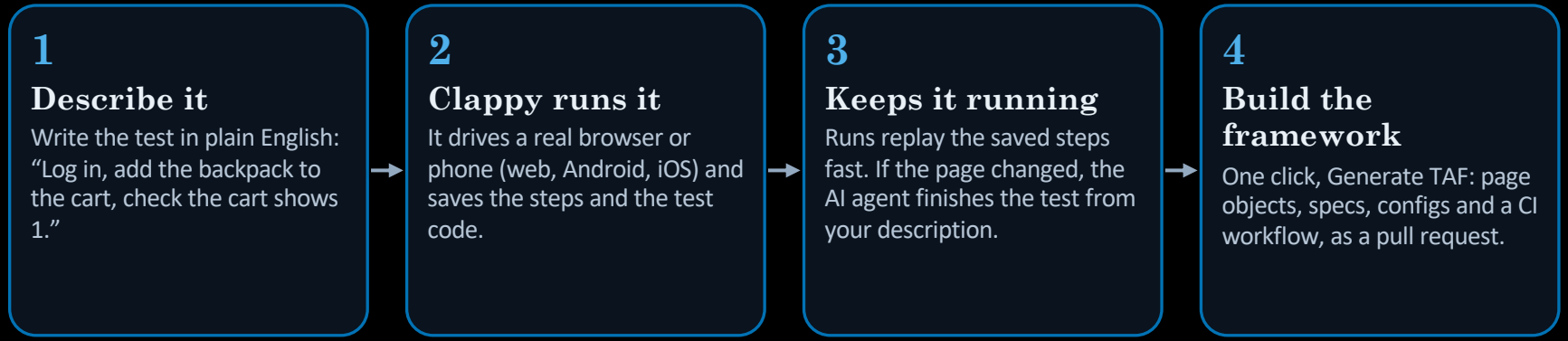


The SDK we built at Applause. It does three jobs:

1. **Writes tests from plain English.** Describe it; it drives a real browser or phone (web, Android, iOS) and saves the code.
2. **Keeps them running.** It replays them and fixes broken locators itself.
3. **Builds your framework.** It turns everyone's tests into one consistent Playwright framework.

From plain English to a test framework with Clappy

In our test platform, testers describe what to test. Clappy does the rest.

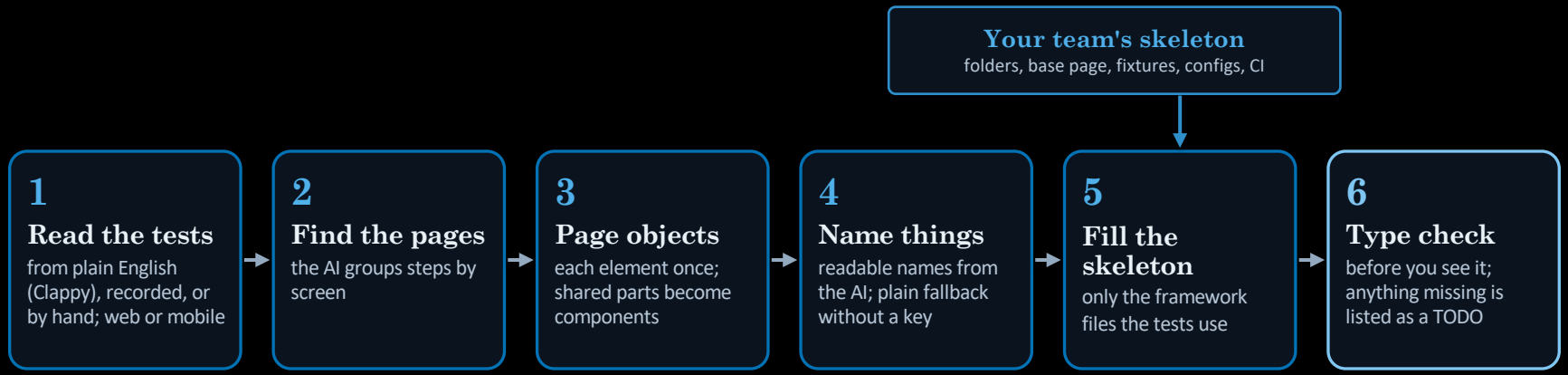


When the app changes: update the test in plain English or regenerate it.

Testers describe the tests; Clappy writes, runs and maintains them, *and builds the framework your team reviews as a pull request.*

How Clappy builds the framework (TAF)

TAF = test automation framework.



Out: page objects, specs, env configs and a CI workflow, as a pull request on your GitHub repo or a zip.

In our test platform
one click: Generate TAF (the demo)

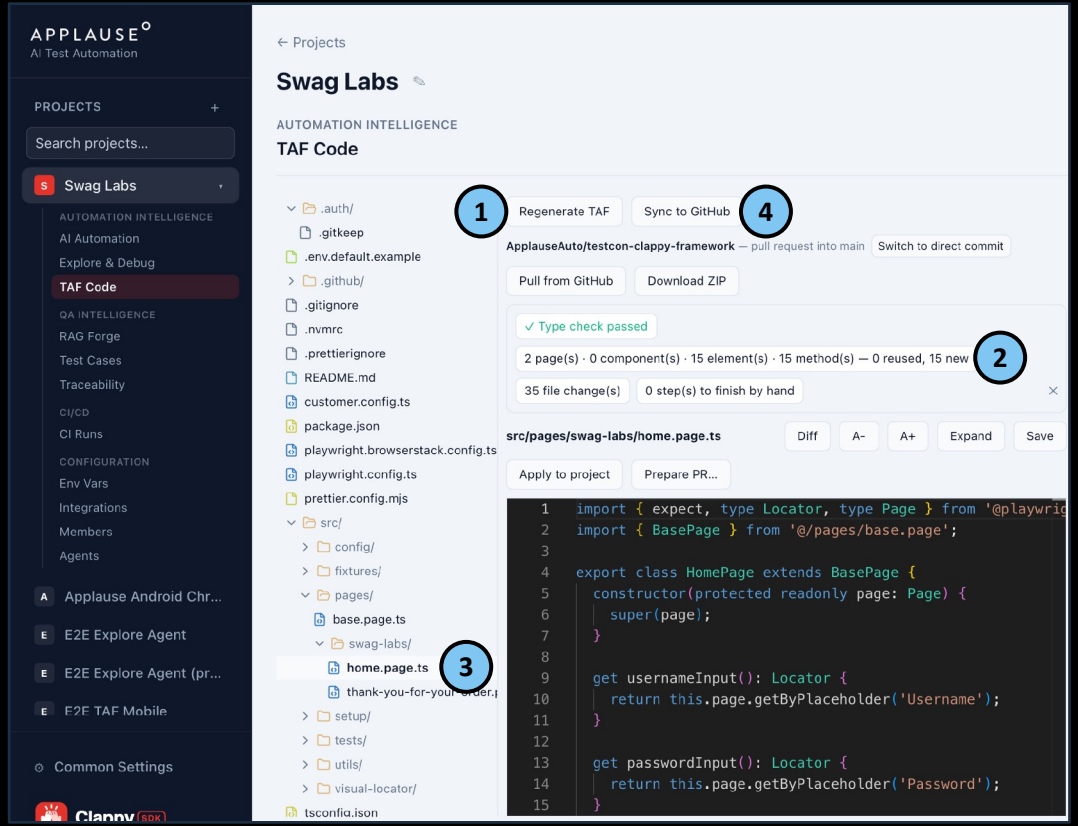
In Claude Code
Claude calls Clappy as a tool (MCP)

In your CI pipeline
a command: clappy generate

In your own scripts
a JavaScript library: clappy-core

The same engine runs in all four places. Clappy is internal to Applause today: it is not open source.

Live demo: one click in our test platform, dispel scepticism



1 **Generate TAF**
Anna's, Ben's and Chris's tests on saucedemo.com go in. About a minute.

2 **Read the report**
A summary of what was built: the pages and elements found, whether the code compiles, and anything left to finish by hand.

3 **Open the page object**
The login (username, password, Login button) is written once, used by all three tests.

4 **Sync to GitHub**
All generated files go to your GitHub repo as one pull request, reviewed like any code.

Three things to remember

1

Give AI your context

Rules to follow, your team's docs, and real tools. Without context, AI can only guess.

`rules + docs + tools`

2

Make it a team standard

Write how you work down once, as rules and skills. Every tester gets the same quality.

`one skill, every tester`

3

Keep a human in the loop

A person reviews every AI change in a pull request, and approves it only with proof that it works.

`proof first, then approve`

Speed is easy. Reliability is the real challenge.

Questions?



Adonis Celestine Global Head of Automation, Applause

Taras Lytvyn AI-first Test Automation Architect, Lead SDET, Applause

Slides, code, RAG setup

github.com/ApplauseAuto/testcon-ai-cc-demo